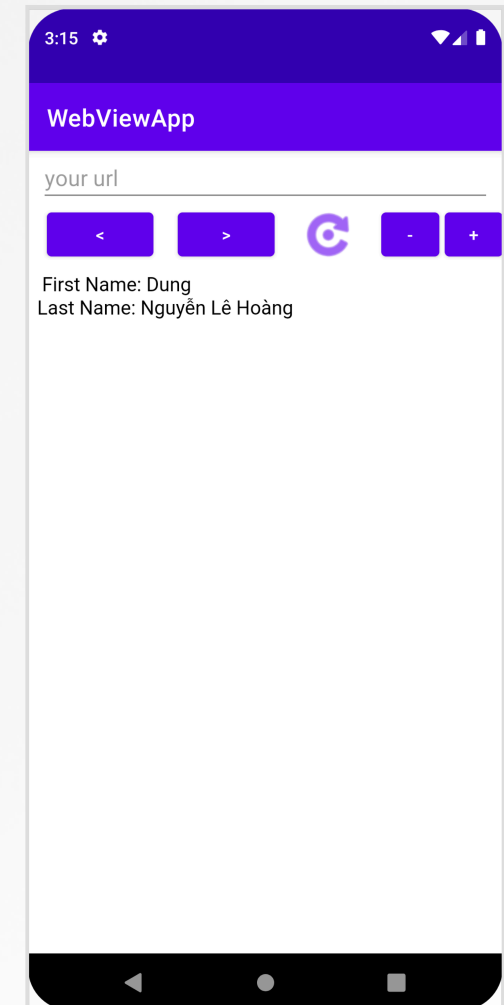


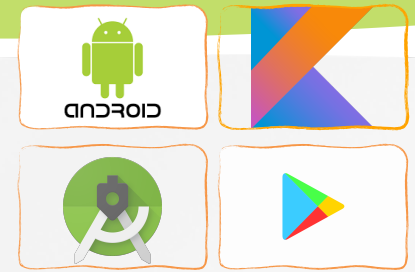
Example 3

Immediate Page

» Main Activity

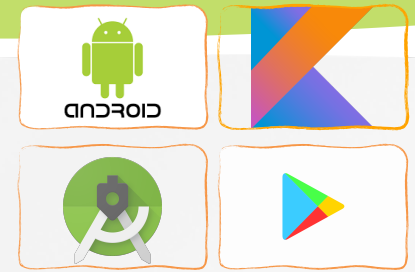
```
fun demo3ImmediatePage(){  
    val htmlString = ""  
        <html>  
            <head> </head>  
            <body>  
                  
                First Name: Dung  
                <br>  
                Last Name: Nguyễn Lê Hoàng  
            </body>  
        </html>  
    """.trimIndent()  
    //1  
    webView.settings.javaScriptEnabled = true  
    //2  
    webView.webViewClient = WebViewClient()  
    //3  
    webView.loadData(htmlString, "text/html", "utf-8")  
}
```





Commonly Used WebView Methods

- » **reload()** refresh the currently viewed web page
- » **goBack()** go back one step in the browser history (use **canGoBack()** to determine if there is any history to trace back)
- » **goForward()** go forward one step in the browser history (use **canGoForward()** to determine if there is any history to go forward to)
- » **goBackOrForward()** go backwards or forwards in the browser history, where negative/positive numbers represent a count of steps to go
- » **canGoBackOrForward()** to see if the browser can go backwards or forwards the stated number of steps.
- » **clearCache()** clear the browser resource cache
- » **clearHistory()** clear the browsing history
- » **ZoomIn()** , **ZoomOut()** magnify control.

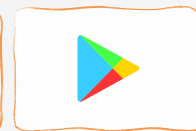


Example 4

Combining HTML & Android

- » The methods in the interface are invoked by the JavaScript code placed behind the HTML side of the app.
- » The interface methods run on the Android side of the app, using Android resources and data stored in the device.





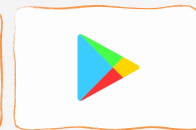
Example 4

Combining HTML & Android

» Create html file

» Add html file to **Assets** folder

```
<!DOCTYPE HTML PUBLIC "-//W3C//DTD HTML 4.01 Transitional//EN">
<html>
<head>
<title>Android GetPhone Demo</title>
<script language="javascript">
function setPerson(nameValue) {
    // move nameValue to input box – reset phone
    document.getElementById("guiName").value = nameValue ;
    document.getElementById("guiPhone").value = "" ;
}
function askAndroidToFindPhone() {
    // call Android, ask it to find the person's phone number
    var person = document.getElementById("guiName").value;
    HtmlAndroidBridge.showToast('Android is busy - Searching phones... ' + person);
    document.getElementById("guiPhone").value =
    HtmlAndroidBridge.getPhoneNumber(person);
}
</script>
```

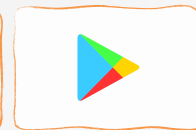


Example 4

Combining HTML & Android

```
<style type="text/css">
    .style1 {
        color: #008000; font-family: Arial, Helvetica, sans-serif;
    }
</style>
</head>

<body>
<h1 class="style1">HTML & Android</h1>
<p><a onClick="setPerson('Dung')">
    <u> Click here to set name to 'Dung'</u></a>
<p><a onClick="setPerson('Kotlin')">
    <u> Click here to set name to 'Kotlin'</u></a>
<p>or just type any person's name
<p> Person's name <input type="text" id="guiName" />
<p> <input type="button" onclick= "askAndroidToFindPhone()" value="Ask Android to
Find Person's Phone Number">
<p> Phone Number <input type="text" id="guiPhone" />
</body>
</html>
```

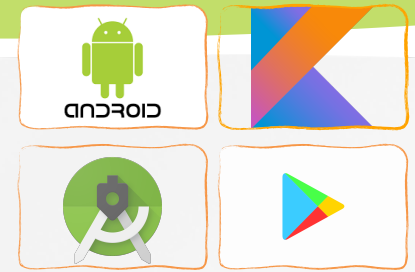


Example 4

Combining HTML & Android

» Create *MyJavaScriptInterface* in Main Activity

```
class MyJavaScriptInterface(val context: Context) {  
    @JavascriptInterface  
    fun showToast(toastMsg: String?) {  
        // instead of dull JavaScript alert() use flashy Toasts  
        Toast.makeText(context, toastMsg, Toast.LENGTH_SHORT).show();  
    }  
  
    @JavascriptInterface  
    fun getPhoneNumber(person: String): String {  
        // accept a person's name and fake the DB processing  
        return if (person == "Dung")  
            "113-1111"  
        else if (person == "Kotlin") "211-1610"  
        else "555-5555"  
    }  
}
```



Example 4

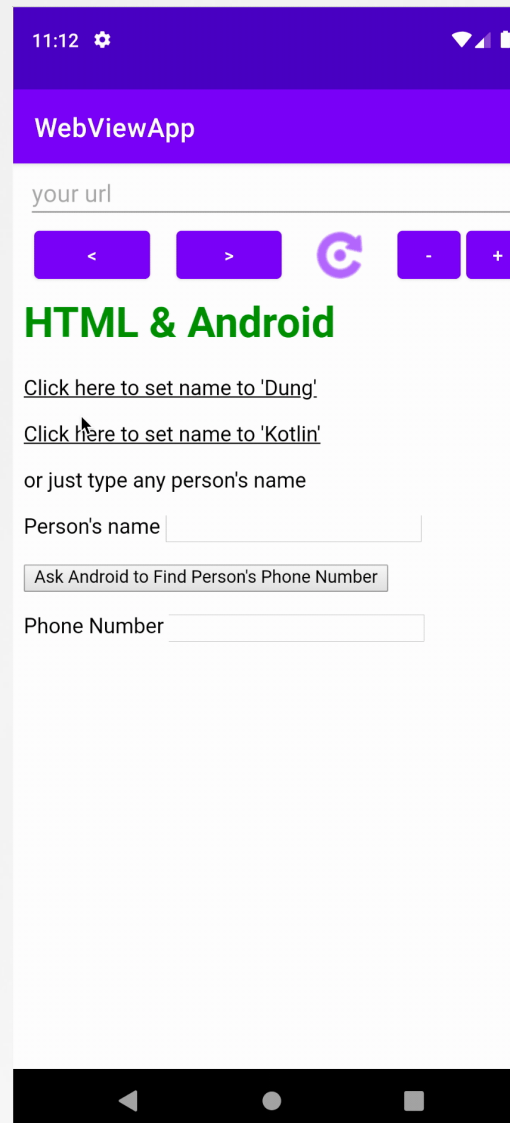
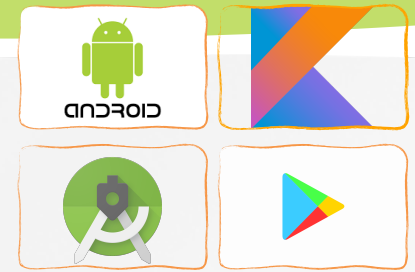
Combining HTML & Android

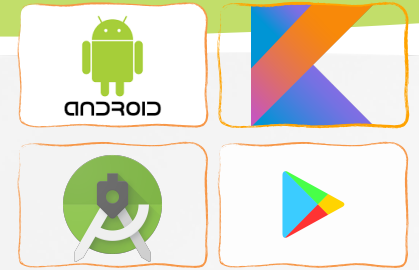
» Main Activity

```
fun demo4LocalPageJavascript(){  
    //1  
    webView.settings.javaScriptEnabled = true  
  
    //2  
    webView.addJavascriptInterface(MyJavaScriptInterface(this), "HtmlAndroidBridge");  
  
    //2  
    webView.loadUrl("file:///android_asset/local_page_javascript.html")  
}
```

Example 4

Combining HTML & Android





Example 4

Combining HTML & Android

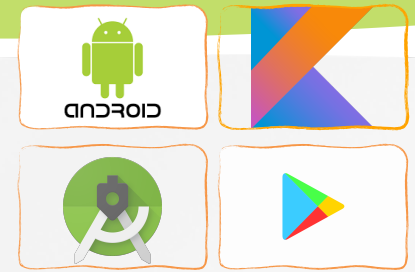
» Comments

- ***MyJavaScriptInterface*** is created as a Kotlin class. An instance of this class is later passed to the JS environment through the statement:

```
webView.addJavascriptInterface(MyJavaScriptInterface(this), "HtmlAndroidBridge");
```

- "[HtmlAndroidBridge](#)" is the alias to be used by JS in html page.
- The JS code behind the HTML pages hosted by the WebView should refer to the methods in the bridge object

```
HtmlAndroidBridge.methodName(params);
```



Example 4

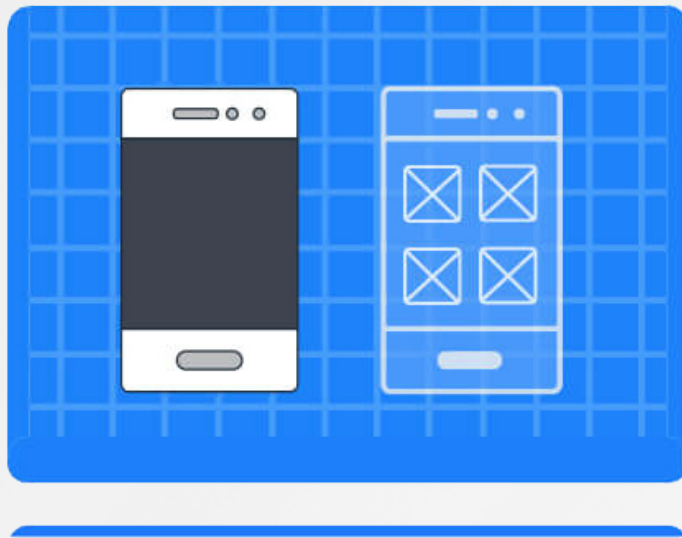
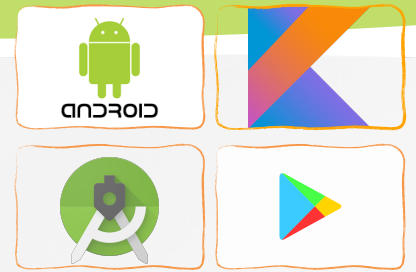
Combining HTML & Android

» Comments

– *MyJavaScriptInterface*

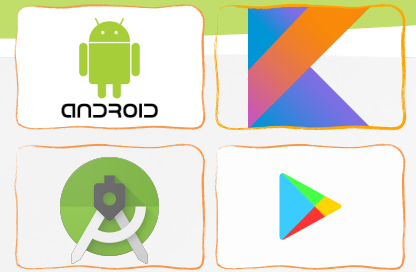
- ***showToast(string)*** will be called by JS code to briefly display an arbitrary message on top of the HTML page held by the WebView. That is an Android object not an HTML resource.
- ***getPhoneNumber(string)*** will be used to fake the Android Contact searching. Databases and Content Providers are studied later in the course.

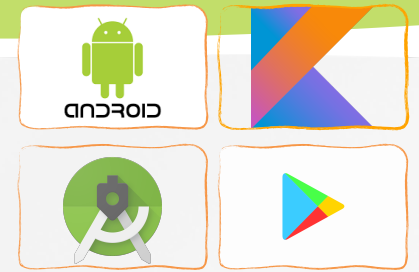
Demo



Kotlin

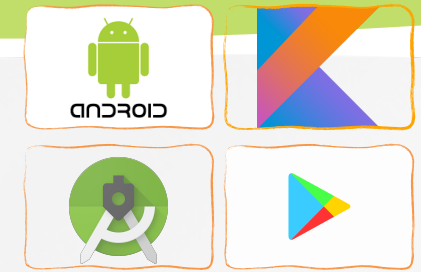
Q&A





Practice 1

- Implement all above examples for using webview widget

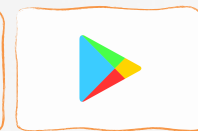


APPENDIX A. Searching for a phone number using the user's Contact-Book

» Add to Manifest

```
<uses-permission android:name="android.permission.READ_CONTACTS" />
```

```
fun requestContacts(){
    if (ContextCompat.checkSelfPermission(context, Manifest.permission.READ_CONTACTS)
        != PackageManager.PERMISSION_GRANTED) {
        // Permission is not granted
        if (ActivityCompat.shouldShowRequestPermissionRationale(context as MainActivity,
            Manifest.permission.READ_CONTACTS)) {
        } else {
            ActivityCompat.requestPermissions(context as MainActivity,
                arrayOf(Manifest.permission.READ_CONTACTS), 11111);
        }
    } else {
        // Permission has already been granted
    }
}
```



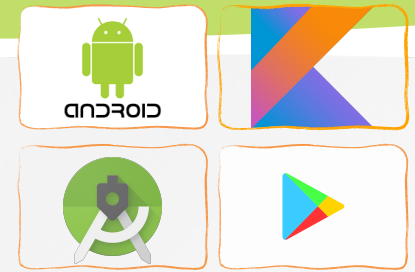
APPENDIX A. Searching for a phone number using the user's Contact-Book

```
// find Phone Number from Contact app
fun findPhoneNumber(searchName: String): String {
    //Request permission
    requestContacts()

    var name = "n.a."
    var number = "n.a."
    //Define an iterator to traverse the contact with Cursor
    val cursor =
context.contentResolver.query(ContactsContract.CommonDataKinds.Phone.CONTENT_URI,
    null, null, null, null)!!

    while (cursor.moveToNext()) {
        val NAME = ContactsContract.CommonDataKinds.Phone.DISPLAY_NAME
        val nameIdx = cursor.getColumnIndex(NAME)
        name = cursor.getString(nameIdx)

        if (name.lowercase().contains(searchName.lowercase())) {
            val NUMBER = ContactsContract.CommonDataKinds.Phone.NUMBER
            val numberIdx = cursor.getColumnIndex(NUMBER)
            number = cursor.getString(numberIdx)
            break
        }
    }
    cursor.close()
    return number
}
```



Reference

- » <https://kotlinlang.org/docs/reference/>
- » Peter Späth, Learn Kotlin for Android Development, 2019
- » Ted Hagos, Learn Android Studio 3 with Kotlin, 2018
- » Peter Späth, Pro Android with Kotlin, 2019
- » Milos Vasic, Mastering Android Development with Kotlin, 2017
- » Victor Matos, Mobile Application Development course, Cleveland State University
- » <https://developer.android.com/guide/>
- » <https://developer.android.com/training/permissions/requesting>